다듬(DaDeum) 완전정복

한국어 맞춤법 교정 엔진 다듬을 **코드를 모르는 사람도** 이해하도록 풀어 쓴 자료 모음입니다. **이 README 하나만 읽어도** 다듬에 쓰인 기술·모델·어려운 말을 전부 알 수 있게 썼습니다.

> 기준 코드: dadeum `master` @ 6af0ea5 (2026-09-21) · 수치는 2026-10-11에 직접 돌려 얻은 값

---

0. 어떤 자료가 있나

읽는 사람	파일	분량
시간이 없는 사람	`다듬-쉽게읽기.pdf`	한 주제 한 쪽, 16주제
단계별로 깊게	`다듬-5단계설명.pdf`	초등학생 → 전문가
질문에 답해야 하는 사람	`다듬-질의응답.pdf`	질문 176개 + 숫자 치트시트
코드를 이해하고 싶은 사람	`다듬-완전정복.pdf`	0~14장 교재
용어가 어려운 사람	이 README의 2~6절 (또는 `다듬-README.pdf`)	아래
로고 후보를 보고 싶은 사람	`design/` (다듬 PR #12의 작업 원본)	후보 4종 · 생성기 · 전체 내보내기

원본: `easy/` `qa/` `chapters/` (Markdown). 다시 만들기: `./build-readme.sh` `./build-easy.sh` `./build-qa.sh` `./build.sh` (pandoc + Chrome 필요)

---

1. 다듬이 뭔가 (30초)

한국어 글을 넣으면 **맞춤법·띄어쓰기·조사·문법 오류에 밑줄**을 긋고, 클릭 한 번에 고쳐 주는 도구입니다. 웹 앱·크롬 확장·macOS 앱 세 가지가 **같은 엔진 하나**를 씁니다. 전부 내 컴퓨터 안에서 돌고 외부 AI·계정·과금이 없습니다.

```
글 입력 → [Kiwi로 문장을 조각내기] → [검사기 21개가 "여기 틀림" 후보를 냄] → [겹치는 후보 정리] → 밑줄
```

핵심 태도: 틀린 걸 못 잡는 것보다, **맞는 글을 틀리게 고치는 것을 훨씬 무겁게** 봅니다. 확신이 없으면 고치지 않고 점선으로만 표시합니다.

---

2. 딥러닝 쓰였나요? (가장 많이 받는 질문)

한 줄 답: 다듬은 **"딥러닝 중심"이 아닙니다.** 쓰인 학습 모델 대부분은 아주 가벼운 **통계·선형 모델**이고, 딥러닝(신경망)은 **작은 것 하나**뿐이며 그마저도 **배포본에서는 빠져 있습니다.**

쓰인 것을 종류별로

종류	이 프로젝트에서	딥러닝?
규칙 (정규식, 조건문)	검사기 21개, 사전 규칙 474개	❌
선형 모델 (TF-IDF + 로지스틱 회귀)	되/돼·안/않·던/든·로서/로써 등 혼동쌍 6개, 단어 경계 판별기	❌
통계 언어모델 (n-gram + 백오프)	gyeol-lm(형태소 3개씩 묶어 센 표), 이상탐지(음절 3개씩)	❌
사전/표 조회	어절 사전 181만 개	❌
신경망 (BiGRU)	띄어쓰기 태거 (`spacing_gru.pt`, 4.8MB)	✅ 작은 규모
사전학습 대형 모델 (BERT, RoBERTa, LLM)	**쓰지 않음** (비교 실험 코드만 있고 배포물에 없음)	—

신경망은 정확히 어디에?

띄어쓰기 태거 하나입니다.

```
구조:  글자 → 64개 숫자(임베딩) → BiGRU 2층(은닉 192, 양방향) → 글자마다 "뒤에 공백이 올 확률"
크기:  파일 4.8MB, 사전학습 없음(처음부터 학습)
용도:  띄어쓰기 "넣기" 후보를 점선(low)으로만 제안 — 자동 교정 권한 없음
배포:  torch 라이브러리가 필요해서 배포 이미지에서 빠짐 (환경에 따라 꺼짐)
```

작은 2층 순환 신경망이라 넓게 보면 딥러닝에 속하지만, “거대 모델”과는 거리가 멉니다.

쓰지 않은 것 (자주 오해하는 부분)

이름	상태
ChatGPT·Gemini 같은 **외부 LLM**	7월에 Gemini 층이 있었으나 **9/15 제거**. 지금은 호출하지 않음
BERT / RoBERTa (`klue/roberta-base`)	띄어쓰기 태거와 **비교하는 실험 코드** (`train_spacing_bert.py`)만 있고 배포·의존성에 없음
KoBART 교정 모델	실제로 설치해 시험했으나 규칙 시스템보다 낮고 새 오류를 만들어 **통합 안 함** (과거 기록)
학습형 문법 교정(편집 태깅 GEC)	7/24에 시도했으나 안전선을 못 넘어 **접음**. 프로토타입 코드는 삭제됨
문장을 통째로 생성하는 모델	없음. 다듬은 **고칠 말을 지어내지 않고** 규칙·사전에서만 고침

---

3. 쓰인 기술 전체 목록

3-1. 모델·알고리즘

이름	쉬운 뜻	다듬에서	장
형태소 분석 (Kiwi)	문장을 뜻의 최소 조각으로 쪼개고 이름표를 붙임	모든 규칙 검사기의 바닥	3
정규식	“이런 모양의 글자를 찾아라”	사전 규칙 474개, 슬롯 찾기	1, 6
글자 n-gram	연속한 2~4글자 조각	혼동쌍·경계 모델의 입력	7
TF-IDF	조각이 얼마나 나왔나 × 얼마나 드문가	조각마다 무게 부여	7
로지스틱 회귀	조각마다 점수를 더해 확률로 바꿈	혼동쌍 6개·경계 판별기	7
시그모이드	아무 수나 0~1 확률로 바꾸는 곡선	로지스틱 회귀의 마지막 단계	7
형태소 n-gram 언어모델	형태소 3개씩 묶은 빈도로 문장이 자연스러운지 점수	gyeol-lm (오타 후보 순위)	9
멍청한 백오프(stupid backoff)	긴 조합이 없으면 짧은 조합으로 내려가 0.4배씩 깎음	gyeol-lm 점수	9
선형 보간	3·2·1글자 확률을 0.70/0.25/0.05로 섞음	이상탐지	9
BiGRU 신경망	글을 앞뒤 양방향으로 읽는 작은 신경망	띄어쓰기 태거	10
해시 + 이진 탐색	단어를 64비트 지문으로 바꿔 정렬해 두고 스무고개로 찾음	어절 사전·언어모델	9

이름	쉬운 뜻	다듬에서	장
diff (SequenceMatcher)	두 글의 차이를 삽입/삭제/교체로 나열	Kiwi 띄어쓰기 결과와 원문 비교	5, 8
키보드 거리	두벌식 자판에서 한 번의 손 실수로 닿는가	오타 검사	9

3-2. 라이브러리·도구

이름	역할	배포에 포함?
Python	엔진 언어	✅
FastAPI	서버 틀(주문 받는 창구)	✅
uvicorn / gunicorn	서버를 실제로 돌리는 프로그램	✅
pydantic	요청·응답 모양 검문	✅
python-dotenv	`.env` 설정 읽기	✅
kiwipiepy (Kiwi)	형태소 분석·띄어쓰기 (LGPL-3.0, **외부 라이브러리**)	✅
scikit-learn	TF-IDF·로지스틱 회귀 **학습**용	❌ 학습 때만
joblib	학습한 모델 저장	❌ 학습 때만
numpy	언어모델 학습 때 계산	❌
torch (PyTorch)	신경망 학습·실행	❌ (선언 안 됨, 환경에 따라)
transformers	BERT 비교 실험	❌ (의존성 아님)
pytest / hypothesis / httpx	테스트 / 무작위 입력 시험 / 테스트용 요청	❌ 개발용
Docker	서버를 상자에 담아 배포	배포 방식
JavaScript (웹·확장)	화면, 프레임워크 없음	✅
Swift + AppKit + 접근성(AX) API	macOS 앱이 다른 앱의 입력창을 읽고 고침	✅ (맥)
Chrome MV3 확장	크롬 확장 규격 3판	✅

3-3. 데이터

이름	쓰임	라이선스
한국어 위키백과	학습·평가용 맞는 문장	CC BY-SA 4.0
nsmc 영화평	구어 학습·평가	CC0
한글 맞춤법 규범 (국립국어원)	규칙의 근거(조항 번호)	규범 인용

---

4. 직접 만든 모델 목록 (결 Gyeol)

“결”은 이 저장소가 **처음부터 학습한** 모델군의 이름입니다. 빌린 것은 학습 알고리즘(scikit-learn)과 코퍼스 원문뿐입니다.

모델	하는 일	방식	파일	상태
gyeol-an	안 / 않	선형	`an.ddm` 5.7MB	✅ 가동 (문턱 0.5)
gyeol-doe	되 / 돼	선형	`doe.ddm` 5.0MB	✅ (0.6)
gyeol-den	던 / 든	선형	`den.ddm` 2.2MB	✅ (0.7)
gyeol-roseo	로서 / 로써	선형	`roseo.ddm` 5.7MB	✅ (0.7, 모델 단독 재현율 20%)
gyeol-reo	러 / 려	선형	`reo.ddm` 5.7MB	⏸ 꺼 둠 (평가 표본 없음)
gyeol-dae	데 / 대	선형	`dae.ddm` 5.7MB	⏸ 꺼 둠
gyeol-boundary	붙여 쓴 말의 단어 경계	선형	`boundary.ddm` 5.7MB	✅ (0.97)
gyeol-lm	형태소 언어모델	n-gram	`lm.bin` 36MB	✅ (순위만)
어절 사전	181만 어절 빈도	해시 표	`lexicon.bin` 22MB	✅
이상탐지	낯선 어절	음절 n-gram	`stats_ko.json` (**저장소에 없음**)	환경에 따라
gyeol-space	띄어쓰기 태거	**BiGRU 신경망**	`spacing_gru.pt` 4.8MB	⏸ torch 필요, 배포 제외

> **가동**이란 `/api/health` 에서 켜진 것으로 확인되는 층입니다. 환경에 따라 달라지므로 결과가 다르면 이 값을 먼저 봅니다.

---

5. 용어 사전 (가나다순)

읽는 법: 용어 — 쉬운 말 → *다듬에서는*

ㄱ

- **가드(guard)** — 오탐이 날 만한 자리를 미리 막는 조건. → *규칙 검사기 코드의 80%가 가드*
- **가중치(weight)** — 조각마다 붙은 "이쪽으로 미는 힘"의 숫자. → *모델 파일의 내용물*
- **검사기(checker)** — 한 가지 오류 부류를 찾는 함수. → *21개*
- **과교정** — 맞는 글을 **틀리게 바꿔 버림**. 오탐 중 가장 나쁜 형태. → *가장 피하는 것*
- **과적합** — 본 문제에는 강하고 새 문제에는 약한 상태. → *dev 74% vs 처음 본 문장 67%*
- **관형사형 어미(ETM)** — 뒤의 명사를 꾸미게 하는 어미(-ㄴ, -는, -던). → `든` *+명사는* `던` *으로 확정*
- **구어 / 문어** — 말투의 글 / 격식 있는 글. → *위키(문어), nsmc(구어)를 따로 잼*
- **규칙 기반** — 사람이 조건을 적어 둔 방식. ↔ 학습 기반

ㄴ

- **n-gram** — 연속한 n개 조각. → *글자 2~4개 / 형태소 3개 / 음절 3개*
- **누수(leakage)** — 시험 문제를 미리 본 것 같은 상태. 성적이 부풀려짐. → *문장 단위로 시험용을 나눠 방지*

ㄷ

- **단위 테스트** — 함수 하나가 맞게 도는지 자동으로 확인. → *1,020개*
- **데코레이터** — 함수 위에 붙여 기능을 더하는 표시(`@router.post`).
- **디바운스** — 입력이 멈출 때까지 기다렸다 한 번만 실행. → *0.25~0.9초*
- **dense(조밀 덩어리)** — 띄어쓰기를 안 한 긴 덩어리(8글자+, 조사·어미 뒤 내용 2회+). → *이 덩어리는 모델 대신 Kiwi를 믿음*

ㄹ

- **라벨(label)** — 학습 데이터의 정답표. → *"쓰인 형태"가 라벨*
- **로지스틱 회귀** — 조각 점수를 더해 확률로 바꾸는 가장 단순한 분류 모델.

ㅁ

- **마스킹** — 정답 자리를 가림(▒). → *모델이 정답을 베끼지 못하게*
- **머신러닝** — 예시에서 판단 기준을 숫자로 익히는 방식.
- **메트릭 / 하한(floor)** — 성능 지표 / 이 아래로 떨어지면 안 되는 선. → *후퇴 금지선*
- **미탐(FN)** — 틀린 걸 놓침.
- **미들웨어** — 요청이 본 로직에 닿기 전에 지나는 관문. → *CORS, 레이트리밋*
- **문턱(threshold)** — 이 확률 이상일 때만 행동하는 기준선.

ㅂ

- **백오프(backoff)** — 긴 조합이 없으면 더 짧은 조합으로 내려감.
- **병합(merge)** — 겹치는 후보를 우선순위로 하나만 남김.
- **보조용언** — 본용언 뒤에서 돕는 용언(-고 있다). → *붙여 써도 맞아서 침묵*
- **불변식** — 어떤 입력에서도 참이어야 하는 성질. → *P1~P5*
- **BiGRU** — 앞뒤 양방향으로 읽는 작은 순환 신경망.

ㅅ

- **서버 / 클라이언트** — 일을 해 주는 쪽 / 시키는 쪽.
- **슬롯(slot)** — 헷갈리는 짝이 나올 수 있는 자리. → `되/돼` *자리*
- **시그모이드** — 아무 수를 0~1로 바꾸는 곡선.
- **신뢰도(confidence)** — high·medium(실선) / low(점선).
- **신경망** — 숫자를 여러 단계로 변환하며 배우는 모델. → *띄어쓰기 태거 하나*
- **어간 / 어미** — 활용하는 말의 변하지 않는 부분 / 뒤에 붙는 문법 조각.

ㅇ

- **API** — “이렇게 주문하면 이렇게 답한다”는 약속. → `POST /api/check`
- **언어모델** — 문장이 자연스러운지 점수를 내는 모델. (ChatGPT 같은 **생성 모델과 다름**) → *gyeol-lm은 점수만, 글을 만들지 않음*
- **어절** — 공백으로 나뉜 덩어리.
- **오탐(FP)** — 맞는 걸 틀렸다고 함.
- **오염 목록** — 정문이라 알았지만 읽어 보니 원문이 틀린 문장을 근거와 함께 뺀 목록.
- **의존명사** — 혼자 못 쓰고 꾸미는 말 뒤에서만 쓰는 명사(것, 수, 뿐).

- **IDF** — 얼마나 드문가. 드물수록 값이 큼.
- **임베딩** — 글자를 숫자 여러 개로 바꾼 것. → *BiGRU 입력*

ㅈ

- **재현율(recall)** — 틀린 것 중 잡아낸 비율.
- **정규식** — 글자 모양을 찾는 작은 언어.
- **정문** — 맞게 쓴 문장.
- **정밀도(precision)** — 고친 것 중 맞은 비율.
- **조사** — 명사 뒤에 붙는 말(을/를, 은/는, 이/가).
- **종성 / 초성 / 중성** — 글자의 받침 / 첫소리 / 가운뎃소리.
- **주석(annotation)** — 사람이 "이 문장의 오류는 이것"이라고 표시하는 일. → *실사용 평가셋*

ㅋ

- **캐시** — 계산한 결과를 저장해 재사용.
- **코퍼스** — 문장을 대량으로 모은 자료. → *위키, nsmc*
- **코드포인트** — 유니코드에서 글자 하나의 번호 단위.
- **커버리지** — 그 구문을 잴 표본이 있는가.
- **Kiwi** — 형태소 분석기. **다듬이 만든 것이 아님.**

ㅌ

- **TF-IDF** — 조각이 얼마나 나왔나(TF) × 얼마나 드문가(IDF).
- **태그(품사 태그)** — 형태소의 종류 이름표(NNG 명사, JKO 목적격 조사 …).
- **태거(tagger)** — 글자마다 표시를 붙이는 모델. → *띄어쓰기 태거*
- **토큰** — 분석기가 돌려주는 조각 하나.

ㅍ

- **퍼즈(fuzz)** — 정문에 무작위로 오류를 넣어 검출률을 보는 시험.
- **파이프라인** — 단계를 차례로 거치는 흐름. → `eval.pipeline`
- **품사** — 단어의 종류(명사, 동사 …).

- **피클(pickle/joblib)** — 파이썬 객체를 통째로 저장하는 형식. → *메모리가 큼, `.ddm`으로 대체*

ㅎ

- **하한 / 상한** — 최소 / 최대 기준선.
- **해시(hash)** — 단어를 고정 길이 지문 숫자로 바꾼 것. → *충돌 시 미탐 방향으로만 틀림*
- **형태소** — 뜻이나 문법 기능이 있는 가장 작은 조각.
- **형태론 게이트** — 문법으로 답이 정해지는 자리를 확률 없이 확정하는 규칙. → *`되` + `고` → `되`*
- **홀드아웃(holdout)** — 학습에 안 쓰고 따로 떼어 둔 시험용.
- **회귀 레지스트리** — 한 번 고친 버그를 영구 등록해 재발을 막는 목록. → *22건*
- **.ddm** — 다듬이 정한 모델 저장 형식(사전·가중치를 이진 탐색으로 읽음).

---

6. 헷갈리기 쉬운 쌍

헷갈리는 것	차이
정밀도 vs 재현율	정밀도 = 말한 것의 정확도, 재현율 = 놓치지 않은 정도
오탐 vs 미탐	오탐 = 맞는 걸 틀렸다 함(위험), 미탐 = 틀린 걸 못 잡음(아쉬움)
언어모델 vs 생성형 AI	다듬의 언어모델은 **점수만** 내고 글을 쓰지 않음
규칙 vs 모델	규칙 = 사람이 조건 적음, 모델 = 예시에서 배움. 다듬은 **규칙이 먼저**
딥러닝 vs 머신러닝	딥러닝은 머신러닝 중 신경망 방식. 다듬의 대부분은 딥러닝이 아닌 통계·선형 모델
직접 만든 vs 빌린 것	모델 가중치는 직접 학습, 형태소 분석(Kiwi)·학습 알고리즘(sklearn)은 빌림
합성 오류 vs 실사용 오류	합성 = 규칙이 넣은 오류(점수 높음), 실사용 = 사람이 쓴 오류(진짜 실력)

---

7. 숫자 한눈에

항목	값
정밀도	99.21%
정문 오탐	0.00%
실사용 자동교정	dev 74% · holdout 72% · holdout2(처음 본 문장) **67%**
검사기 / 출처 / 사전 규칙	21 / 58 / 474
단위 테스트	1,020
어절 사전	1,806,050
개발 기간 / 커밋	2026-07-14~09-21 / 377

> README(원 프로젝트)는 holdout2를 63%로 적고 있고 제 환경 실측은 67%입니다. 원인은 확인하지 못했습니다.

---

8. 말하면 안 되는 것

- 다른 제품과의 성능 **비교 수치**(이 자료에 근거 없음)
- “사용자 N명이 쓴다”(사용자 기록 없음)
- “100% 정확” / “오탐이 전혀 없다”(측정한 9,765문장에서만 0)
- “전부 직접 만들었다”(Kiwi는 외부 라이브러리)
- “딥러닝/AI 기반”이라는 포괄 표현(대부분 규칙과 선형·통계 모델)
- “크롬 확장이 완벽히 동작한다”(실제 Chrome 재확인 필요)